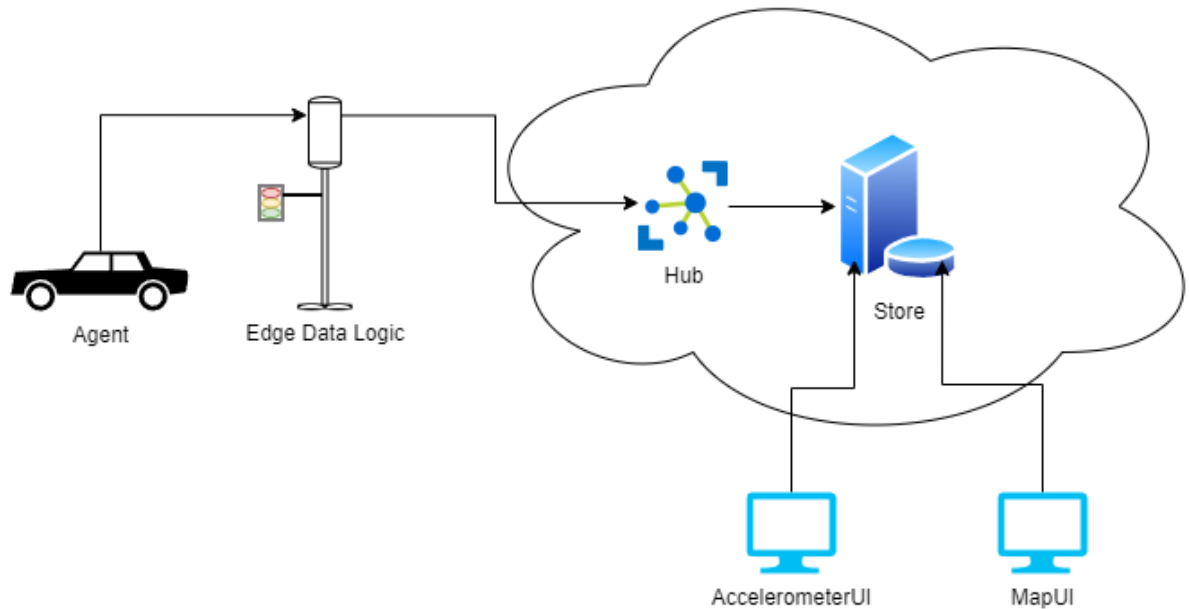


Лабораторна робота №4

Опис проекту



Завдання – розробити комплексну систему моніторингу та аналізу стану дорожнього покриття. Система може мати можливість збирати дані про різні параметри, такі як нерівності, тріщини, вибоїни та інші дефекти, які можуть вплинути на якість дороги. Зібрані дані слід проаналізувати, щоб визначити закономірності та тенденції, які можуть допомогти у прогнозуванні потреб у технічному обслуговуванні дорожньої мережі. Система також може мати можливість генерувати звіти та сповіщення для відповідних органів влади та/або водіїв.

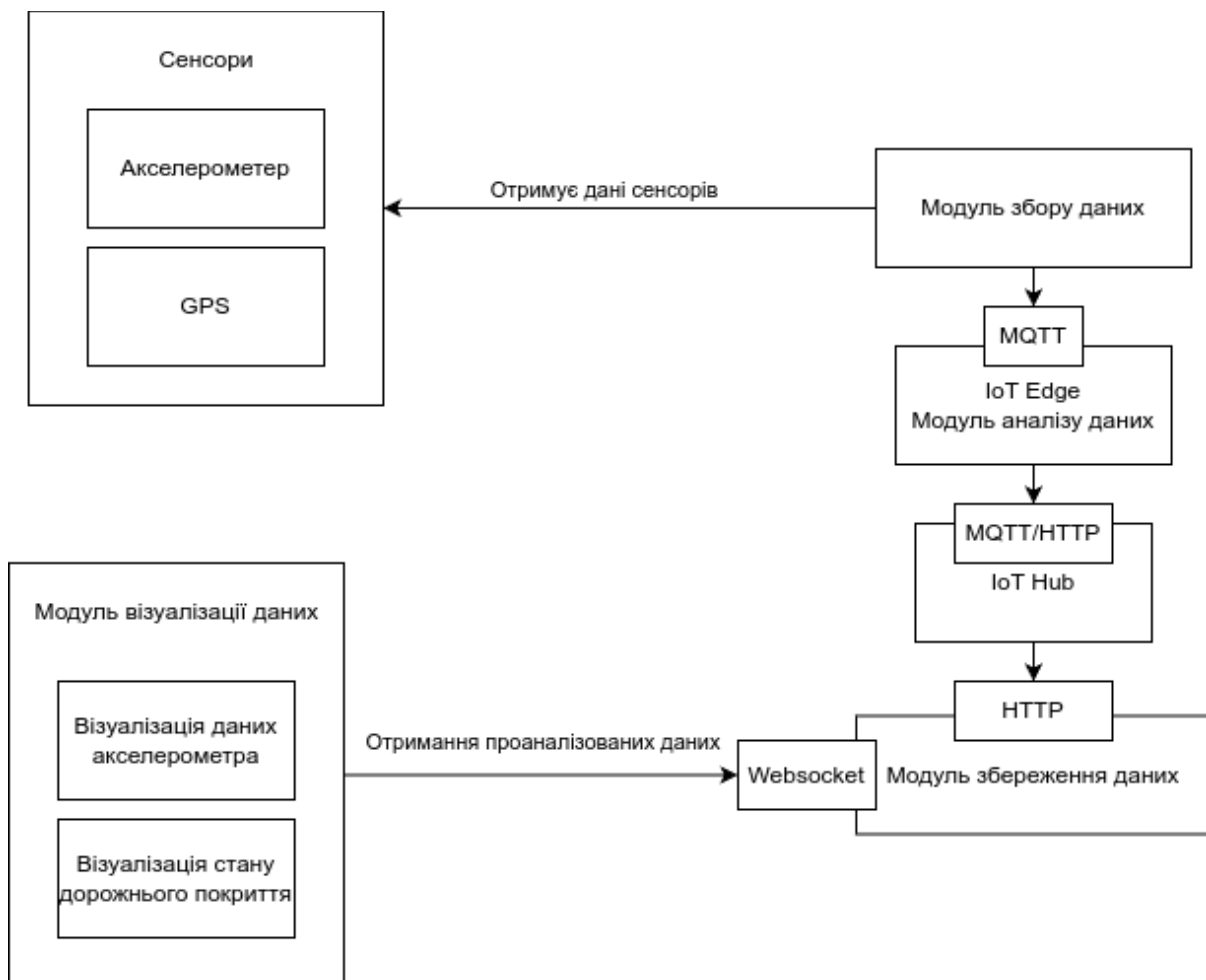
Метою цієї системи є підвищення безпеки дорожнього руху, зниження витрат на технічне обслуговування та покращення загального досвіду водіння для автомобілістів.

В систему входять пристрій на машині (stm32), edge пристрій, хмарні або on-prem сервіси.

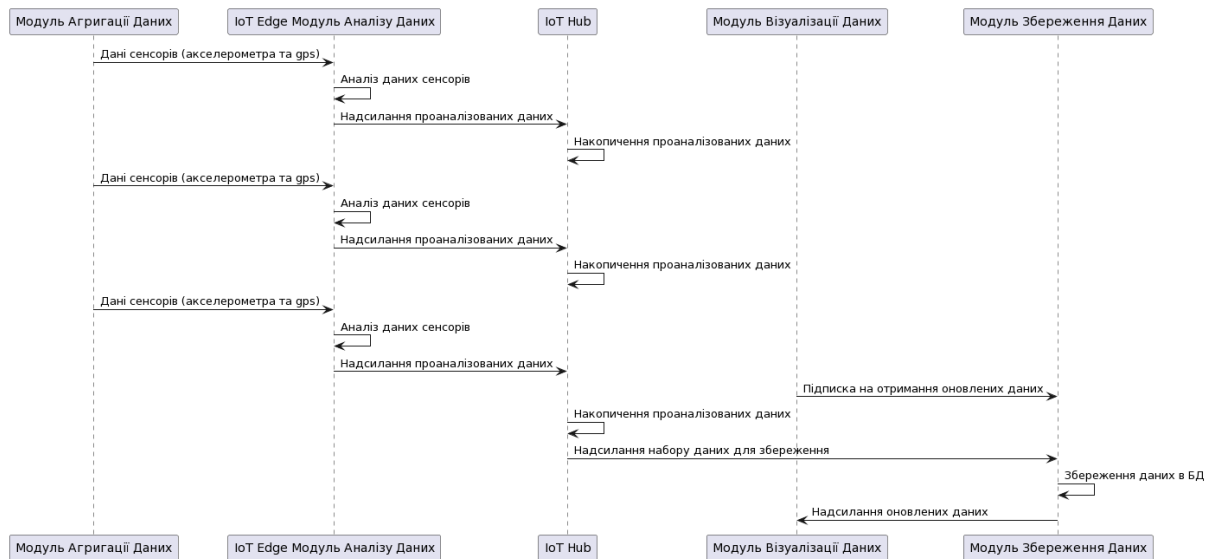
Система містить наступні частини (дозволяється розширювати):

- Модуль збору даних (Agent) - програма агент на машині, яка знімає показники сенсорів (наприклад раз в 100мс) та відправляє їх на сервер.

- Модуль первинної обробки даних (Edge Data Logic) - програма яка займається первинною обробкою даних, в нашому випадку визначає стан дорожнього покриття та відправляє проаналізовані дані на Hub.
- Модуль накопичення даних (Hub) - сервіс який займається накопиченням та аналізом (preprocessing) отриманих даних від Agent та подальшим зберіганням оброблених даних в БД. Перед збереженням дани потрібно накопичити (наприклад 10 записів), так як такі дані простіше аналізувати і запис в БД множини даних буде швидшим.
- Store - арі для отримання та збереження даних в БД.
- Модуль візуалізації (Accelerometer UI, Map UI) - інтерфейс візуалізації (моніторингу) результатів системи.



Архітектура системи



Юзкейс роботи

Постановка задачі

Потрібно реалізувати Edge Data Logic. А саме, дана частина займається аналізом даних з датчиків на машині. Потрібно реалізувати логіку збору даних з MQTT, аналізу стану дорожнього покриття, та відправки проаналізованих даних на Hub. Наприклад за умови, якщо дані по осі Y більше за певне значення, то тоді в нас є яма.

Кроки виконання

Надалі буде показана реалізація на python, проте ніхто не забороняє використовувати той технологічний стек який для вас є оптимальним.

До початку роботи потрібно встановити [Docker](#) та [MQTT Explorer](#).

1. Створення віртуального середовища. Пакет **virtualenv** повинен бути встановлений в системі.

```
python -m venv venv
```

2. Активація середовища

```
source ./venv/bin/activate # linux  
.\venv\Scripts\activate.bat # windows
```

3. Встановимо необхідні залежності(бібліотеки)

```
pip install paho-mqtt # mqtt client  
pip install marshmallow # for schema and converting object to json
```

4. Збережемо залежності в requirements.txt

```
pip freeze > requirements.txt
```

5. Структура проекту буде наступною

```
main.py  
config.py  
app  
    adapters  
        agent_mqtt_adapter.py  
        hub_http_adapter.py  
        hub_mqtt_adapter.py  
    entities  
        agent_data.py  
        processed_agent_data.py  
    interfaces  
        agent_gateway.py  
        hub_gateway.py  
    usecases  
        data_processing.py  
docker  
    docker-compose.yaml  
Dockerfile  
requirements.txt
```

README.md

6. Моделі даних беремо з попередніх лабораторних робіт
7. Визначаємо інтерфейси

agent_gateway.py

```
1  from abc import ABC, abstractmethod
2
3
4  class AgentGateway(ABC):
5      """
6      Abstract class representing the Agent Gateway interface.
7      All agent gateway adapters must implement these methods.
8      """
9
10     @abstractmethod
11     def on_message(self, client, userdata, msg):
12         """
13         Method to handle incoming messages from the agent.
14         Parameters:
15             client: MQTT client instance.
16             userdata: Any additional user data passed to the MQTT client.
17             msg: The MQTT message received from the agent.
18         """
19         pass
20
21     @abstractmethod
22     def connect(self):
23         """
24         Method to establish a connection to the agent.
25         """
26         pass
27
28     @abstractmethod
29     def start(self):
30         """
31         Method to start listening for messages from the agent.
32         """
33         pass
34
35     @abstractmethod
36     def stop(self):
37         """
38         Method to stop the agent gateway and clean up resources.
39         """
40         pass
```

hub_gateway.py

```
1  from abc import ABC, abstractmethod
2  from app.entities.processed_agent_data import ProcessedAgentData
3
4
5  class HubGateway(ABC):
6      """
7      Abstract class representing the Store Gateway interface.
8      All store gateway adapters must implement these methods.
9      """
10
11     @abstractmethod
12     def save_data(self, processed_data: ProcessedAgentData) -> bool:
13         """
14         Method to save the processed agent data in the database.
15         Parameters:
16             processed_data (ProcessedAgentData): The processed agent data to be saved.
17         Returns:
18             bool: True if the data is successfully saved, False otherwise.
19         """
20     pass
```

8. Визначимо конфігурації

config.py

```

1  import os
2
3
4  def try_parse_int(value: str):
5      try:
6          return int(value)
7      except Exception:
8          return None
9
10
11 # Configuration for agent MQTT
12 MQTT_BROKER_HOST = os.environ.get("MQTT_BROKER_HOST") or "localhost"
13 MQTT_BROKER_PORT = try_parse_int(os.environ.get("MQTT_BROKER_PORT"))
14 or 1883
15 MQTT_TOPIC = os.environ.get("MQTT_TOPIC") or "agent_data_topic"
16
17 # Configuration for hub MQTT
18 HUB_MQTT_BROKER_HOST = os.environ.get("HUB_MQTT_BROKER_HOST") or
19 "localhost"
20 HUB_MQTT_BROKER_PORT =
21 try_parse_int(os.environ.get("HUB_MQTT_BROKER_PORT")) or 1883
22 HUB_MQTT_TOPIC = os.environ.get("HUB_MQTT_TOPIC") or
23 "processed_agent_data_topic"
24
25 # Configuration for hub HTTP
26 HUB_HOST = os.environ.get("HUB_HOST") or "localhost"
27 HUB_PORT = try_parse_int(os.environ.get("HUB_PORT")) or 8000
28 HUB_URL = f"http://{HUB_HOST}:{HUB_PORT}"

```

9. Студент реалізовує data_processing.py. Реалізація функції process_agent_data потрібно на базі даних з акселерометра визначити стан дорожнього портиння. Наприклад класифікація може відбуватись через перевірку в якому діапазоні знаходиться координата з акселерометра.

10.

data_processing.py

```

1  from app.entities.agent_data import AgentData
2  from app.entities.processed_agent_data import ProcessedAgentData
3
4
5  def process_agent_data(
6      agent_data: AgentData,
7  ) -> ProcessedAgentData:
8      """
9      Process agent data and classify the state of the road surface.
10     Parameters:
11         agent_data (AgentData): Agent data that contains accelerometer, GPS, and timestamp.
12     Returns:
13         processed_data (ProcessedAgentData): Processed data containing the classified state of
14     the road surface and agent data.
15     """

```

11. Реалізуємо основний файл для запуску програми
main.py


```

1  import logging
2  from app.adapters.agent_mqtt_adapter import AgentMQTTAdapter
3  from app.adapters.hub_http_adapter import HubHttpAdapter
4  from app.adapters.hub_mqtt_adapter import HubMqttAdapter
5  from config import MQTT_BROKER_HOST, MQTT_BROKER_PORT, MQTT_TOPIC,
6  HUB_URL, HUB_MQTT_BROKER_HOST, HUB_MQTT_BROKER_PORT, HUB_MQTT_TOPIC
7
8  if __name__ == "__main__":
9      # Configure logging settings
10     logging.basicConfig(
11         level=logging.INFO, # Set the log level to INFO (you can use logging.DEBUG for
12 more detailed logs)
13         format="[%asctime)s] [%levelname)s] [%module)s] %(message)s",
14         handlers=[
15             logging.StreamHandler(), # Output log messages to the console
16             logging.FileHandler("app.log"), # Save log messages to a file
17         ],
18     )
19     # Create an instance of the StoreApiAdapter using the configuration
20     # hub_adapter = HubHttpAdapter(
21     #     api_base_url=HUB_URL,
22     # )
23     hub_adapter = HubMqttAdapter(
24         broker=HUB_MQTT_BROKER_HOST,
25         port=HUB_MQTT_BROKER_PORT,
26         topic=HUB_MQTT_TOPIC,
27     )
28     # Create an instance of the AgentMQTTAdapter using the configuration
29     agent_adapter = AgentMQTTAdapter(
30         broker_host=MQTT_BROKER_HOST,
31         broker_port=MQTT_BROKER_PORT,
32         topic=MQTT_TOPIC,
33         hub_gateway=hub_adapter,
34     )
35     try:
36         # Connect to the MQTT broker and start listening for messages
37         agent_adapter.connect()
38         agent_adapter.start()
39         # Keep the system running indefinitely (you can add other logic as needed)
40         while True:
41             pass
42     except KeyboardInterrupt:
43         # Stop the MQTT adapter and exit gracefully if interrupted by the user
44         agent_adapter.stop()
45         logging.info("System stopped.")

```

12.Добавимо Dockerfile для запуска данного агента в docker.

Dockerfile

```
1  # set base image (host OS)
2  FROM python:latest
3  # set the working directory in the container
4  WORKDIR /usr/agent
5  # copy the dependencies file to the working directory
6  COPY requirements.txt .
7  # install dependencies
8  RUN pip install -r requirements.txt
9  # copy the content of the local src directory to the working directory
10 COPY src/ .
11 # command to run on container start
12 CMD ["python", "main.py"]
```

13. Налаштуємо `docker-compose.yml` для локального запуску всіх частин та тестування програми.

`docker-compose.yml`

```
1  version: "3.9"
2  name: "road_vision"
3  services:
4    mqtt:
5      image: eclipse-mosquitto
6      container_name: mqtt
7      volumes:
8        - ./mosquitto:/mosquitto
9        - ./mosquitto/data:/mosquitto/data
10       - ./mosquitto/log:/mosquitto/log
11     ports:
12       - 1883:1883
13       - 9001:9001
14     networks:
15       mqtt_network:
16
17
18   postgres_db:
19     image: postgres:latest
20     container_name: postgres_db
21     restart: always
22     environment:
23       POSTGRES_USER: user
24       POSTGRES_PASSWORD: pass
25       POSTGRES_DB: test_db
26     volumes:
27       - postgres_data:/var/lib/postgresql/data
28       - ./db/structure.sql:/docker-entrypoint-initdb.d/structure.sql
29     ports:
30       - "5432:5432"
31     networks:
32       db_network:
33
34
35   pgadmin:
36     container_name: pgadmin4
37     image: dpage/pgadmin4
38     restart: always
39     environment:
40       PGADMIN_DEFAULT_EMAIL: admin@admin.com
41       PGADMIN_DEFAULT_PASSWORD: root
42     volumes:
```

```
43     - pgadmin-data:/var/lib/pgadmin
44     ports:
45     - "5050:80"
46     networks:
47     db_network:
48
49
50     redis:
51     image: redis:latest
52     container_name: redis
53     ports:
54     - "6379:6379"
55     networks:
56     hub_redis:
57
58
59     store:
60     container_name: store
61     build: ../store
62     depends_on:
63     - postgres_db
64     restart: always
65     environment:
66     POSTGRES_USER: user
67     POSTGRES_PASSWORD: pass
68     POSTGRES_DB: test_db
69     POSTGRES_HOST: postgres_db
70     POSTGRES_PORT: 5432
71     ports:
72     - "8000:8000"
73     networks:
74     hub_store:
75     db_network:
76     hub:
77
78
79     edge:
80     container_name: edge
81     build: ../edge
82     depends_on:
83     - mqtt
84     - hub
```

```
85     environment:
86         MQTT_BROKER_HOST: "mqtt"
87         MQTT_BROKER_PORT: 1883
88         MQTT_TOPIC: "agent_data_topic"
89         HUB_HOST: "hub"
90         HUB_PORT: 8000
91         HUB_MQTT_BROKER_HOST: "mqtt"
92         HUB_MQTT_BROKER_PORT: 1883
93         HUB_MQTT_TOPIC: "processed_data_topic"
94     networks:
95         mqtt_network:
96         edge_hub:
97
98
99     hub:
100         container_name: hub
101         build: ../hub
102         depends_on:
103             - mqtt
104             - redis
105             - store
106         environment:
107             STORE_API_HOST: "store"
108             STORE_API_PORT: 8000
109             REDIS_HOST: "redis"
110             REDIS_PORT: 6379
111             MQTT_BROKER_HOST: "mqtt"
112             MQTT_BROKER_PORT: 1883
113             MQTT_TOPIC: "processed_data_topic"
114         ports:
115             - "9000:8000"
116         networks:
117             mqtt_network:
118             hub_store:
119             hub_redis:
120
121
122     fake_agent:
123         container_name: agent
124         build: ../agent
125         depends_on:
126             - mqtt
```

```
127     environment:
128         MQTT_BROKER_HOST: "mqtt"
129         MQTT_BROKER_PORT: 1883
130         MQTT_TOPIC: "agent_data_topic"
131         DELAY: 0.1
132     networks:
133         mqtt_network:
134
135
136 networks:
137     mqtt_network:
138     db_network:
139     edge_hub:
140     hub:
141     hub_store:
142     hub_redis:
143
144
145 volumes:
146     postgres_data:
147     pgadmin-data:
148
```

14.Запуск програму в docker

`cd docker`

`docker-compose up --build`

15.Перевірка працездатності в MQTT Explorer, PgAdmin

Очікувані результати в протоколі

1. Посилання на git репозиторій з проектом
2. Реалізація AgentMqttAdapter, data_processing, HubAdapter
3. Показ результатів роботи в MQTT Explorer та PgAdmin
4. Висновки

